



Relatório de Qualidade — Sistema Aerocode

Disciplina: Técnicas de Programação

Curso: 2º DSM — FATEC

Projeto: Aerocode — Sistema de Gerenciamento de Produção de Aeronaves

Autor: Pedro Lucas dos Santos

Introdução

O presente relatório descreve o desenvolvimento e a avaliação de qualidade da aplicação web **Aerocode**, um sistema crítico de gerenciamento da produção de aeronaves. O sistema foi desenvolvido como continuidade das etapas anteriores do projeto, partindo de uma versão de linha de comando (CLI) para uma aplicação web completa com interface gráfica e back-end integrado ao banco de dados.

Uma aplicação web, diferente de um site tradicional, vai além da exibição de conteúdo: ela permite a interação do usuário, o processamento de dados, a autenticação de acessos e o armazenamento de informações em bancos de dados. O Aerocode exemplifica esse conceito ao oferecer funcionalidades como cadastro de aeronaves, controle de etapas de produção, registro de testes e geração de relatórios de entrega.

Por tratar-se de um sistema crítico — cujas falhas podem impactar diretamente o processo de fabricação de aeronaves — a qualidade do sistema não se limita à correteude funcional. É necessário também mensurar e comprovar seu desempenho sob condições reais de uso. Para isso, foram coletadas métricas de latência, tempo de processamento e tempo de resposta, apresentadas neste relatório com seus respectivos gráficos.

Tecnologias Utilizadas

O desenvolvimento seguiu as recomendações da consultoria contratada, priorizando tecnologias livres, gratuitas, de código aberto, robustas e amplamente utilizadas na indústria.

Back-end

Node.js 22+ - Plataforma de execução JavaScript no servidor .

TypeScript 5.7 - Tipagem estática e segurança no desenvolvimento.

Express.js 4.21 - Framework HTTP para criação da API REST.

Prisma ORM 6.0 - Mapeamento objeto-relacional com o banco de dados.

MySQL 8.0 - Sistema gerenciador de banco de dados relacional.

Zod 3.23 - Validação de dados de entrada.

JSON Web Token (JWT) 9.0 - Autenticação e autorização de usuários.

bcrypt 5.1 - Criptografia de senhas.

Docker 29.3 - Containerização do banco de dados MySQL.

Front-end

Reactb19 - Biblioteca para construção de interfaces.

Vite 8 - Ferramenta de build e servidor de desenvolvimento.

TypeScript 6.0 - Tipagem estática no front-end.

Tailwind CSS 4.2 - Estilização da interface.

React Router DOM 7 - Navegação entre páginas.

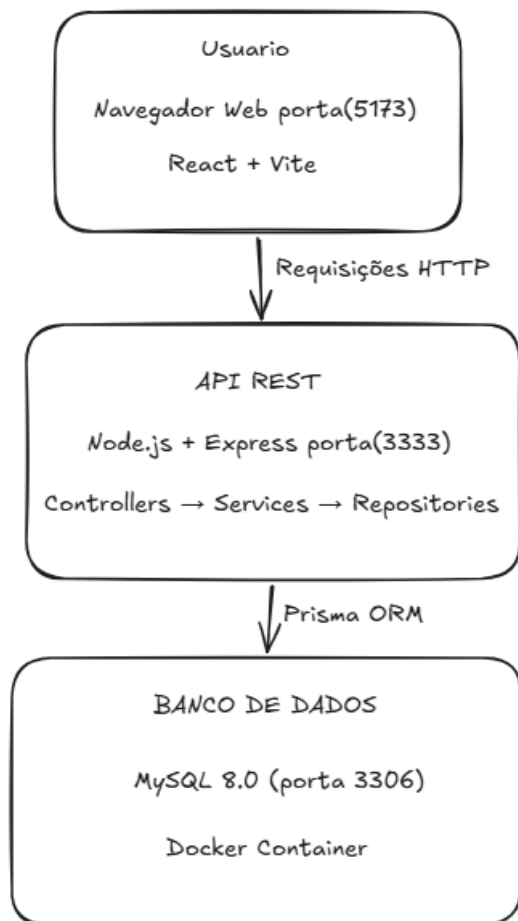
Ferramentas de Qualidade

Autocannon - Simulação de carga e medição do tempo de resposta.

process.hrtime.bigint() - Medição precisa do tempo de processamento no servidor.

Arquitetura da Aplicação

A aplicação Aerocode é composta por três camadas independentes que se comunicam entre si:



Organização do Back-end

O back-end foi desenvolvido seguindo princípios de Clean Architecture e SOLID, com separação clara de responsabilidades:

Controllers: Recebem as requisições HTTP e delegam para os services. Não contêm lógica de negócio.

Services: Camada onde residem todas as regras de negócio do sistema.

Repositories: Centralizam o acesso ao banco de dados via Prisma ORM.

Middlewares: Tratam autenticação, autorização, validação e erros globalmente.

Schemas: Definem e validam os dados de entrada com Zod.

Segurança

- Autenticação via JWT (JSON Web Token)
- Senhas armazenadas com hash bcrypt (custo 10)
- Hierarquia de permissões: OPERADOR < ENGENHEIRO < ADMINISTRADOR
- Proteção de headers via Helmet
- Controle de origem via CORS
- Rate limiting: máximo de 200 requisições a cada 15 minutos

Banco de Dados

O banco de dados possui 9 tabelas com relacionamentos definidos por chaves estrangeiras:

Funcionários: Usuários do sistema com níveis de permissão.

Aeronaves: Aeronaves cadastradas para produção.

Pecas: Peças vinculadas a cada aeronave.

Etapas: Etapas do processo de produção.

etapa_funcionarios: Associação entre etapas e funcionários (N:N).

testes: Testes elétricos, hidráulicos e aerodinâmicos.

relatórios: Relatórios finais de entrega.

ordens_fabricacao: Ordens de fabricação vinculadas a aeronaves.

ordem_pecas: Peças associadas a cada ordem (N:N com quantidade).

ordem_pecas: Peças associadas a cada ordem (N:N com quantidade).

Endpoints da API

A API segue padrão REST com versionamento na URL (`/api/v1`). Os principais grupos de endpoints são:

Grupo	Prefixo	Descrição
Autenticação	`/auth`	Login e dados do usuário logado
Funcionários	`/funcionarios`	CRUD de usuários do sistema
Aeronaves	`/aeronaves`	CRUD de aeronaves
Peças	`/aeronaves/:codigo/pecas`	Gestão de peças por aeronave
Etapas	`/aeronaves/:codigo/etapas`	Controle de etapas de produção
Testes	`/aeronaves/:codigo/testes`	Registro de testes técnicos
Relatórios	`/aeronaves/:codigo/relatorios`	Geração de relatórios de entrega
Ordens	`/ordens`	Ordens de fabricação

Métricas de Qualidade

Para comprovar a qualidade e o desempenho da aplicação, foram coletadas três métricas de acordo com a metodologia descrita a seguir.

Definição das Métricas

Latência: Tempo de transmissão de dados pela rede entre o cliente e o servidor. Representa o atraso natural da comunicação, independente do processamento. Calculada como a diferença entre o tempo de resposta total (medido pelo cliente) e o tempo de processamento (medido pelo servidor).

Tempo de Processamento: Tempo que o servidor levou para processar a requisição desde o momento do recebimento até o envio da resposta. Mede exclusivamente o trabalho realizado pelo servidor, sem considerar a rede.

Tempo de Resposta: Tempo total percebido pelo usuário desde o envio da requisição até o recebimento completo da resposta. É a soma da latência de rede e do tempo de processamento, sendo o indicador mais próximo da experiência real do usuário.

Tempo de Resposta = Latência + Tempo de Processamento

Metodologia de Medição

A coleta das métricas foi realizada com dois mecanismos complementares:

Medição do Tempo de Processamento (lado servidor):

Foi implementado um middleware no framework Express que utiliza a função “process.hrtime.bigint()” do Node.js para capturar o instante de chegada de cada requisição e o instante de envio da resposta, com precisão em nanosegundos. A diferença entre os dois instantes, convertida para milissegundos, representa o tempo de processamento. Todos os valores são armazenados em memória e disponibilizados via endpoint `/api/v1/metrics``.

```
const inicio = process.hrtime.bigint();

res.on('finish', () => {

  const fim = process.hrtime.bigint();

  const ms = Number(fim - inicio) / 1_000_000; // nanosegundos → ms

  metricsStore.registrar({ tempoProcessamentoMs: ms, ... });

});
```

Medição do Tempo de Resposta (lado cliente):

Foi utilizada a biblioteca **autocannon**, uma ferramenta de teste de carga para Node.js que simula múltiplos usuários fazendo requisições simultâneas à API. O autocannon mede o tempo total de round-trip de cada requisição no lado do cliente, fornecendo valores de média, percentil 50 (p50), percentil 95 (p95) e percentil 99 (p99).

Cálculo da Latência:

A latência foi calculada subtraindo o tempo médio de processamento (medido pelo servidor) do tempo médio de resposta (medido pelo cliente):

Latência = Tempo de Resposta - Tempo de Processamento

Configuração dos testes:

- Endpoint testado: `GET /api/v1/aeronaves`
- Duração de cada teste: 10 segundos
- Cenários: 1, 5 e 10 conexões simultâneas
- Unidade de medida: milissegundos (ms)
- Ambiente: localhost (Windows 11, Node.js 25.6.1, MySQL 8.0 via Docker)

Resultados

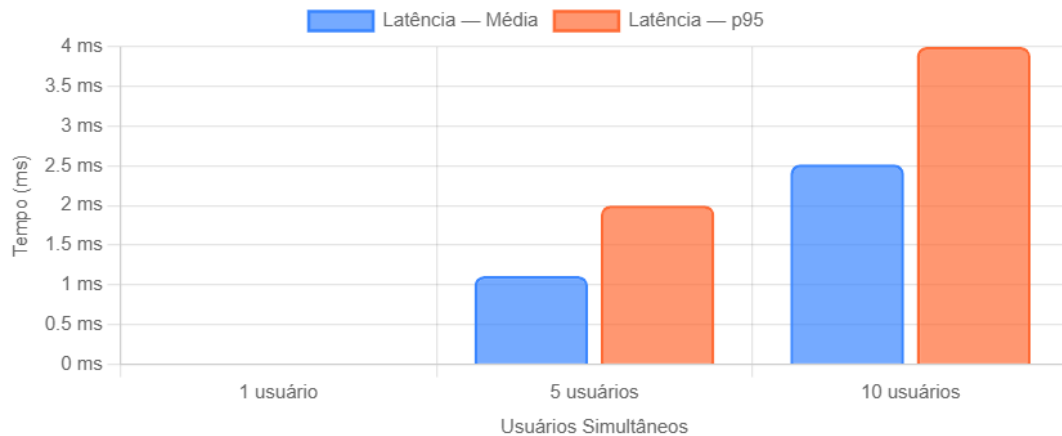
A tabela abaixo apresenta os valores coletados nos três cenários de teste:

Usuários	Latência Média (ms)	Latência p95 (ms)	Processamento Médio (ms)	Processamento p95 (ms)	Resposta Média (ms)	Resposta p95 (ms)	Req/s	Erros
1	0,01	0,00	< 0,01	< 0,01	0,01	0,00	5.793	0
5	1,11	2,00	< 0,01	< 0,01	1,11	2,00	3.118	0
10	2,52	4,00	< 0,01	< 0,01	2,52	4,00	3.217	0

Gráfico 1 — Latência

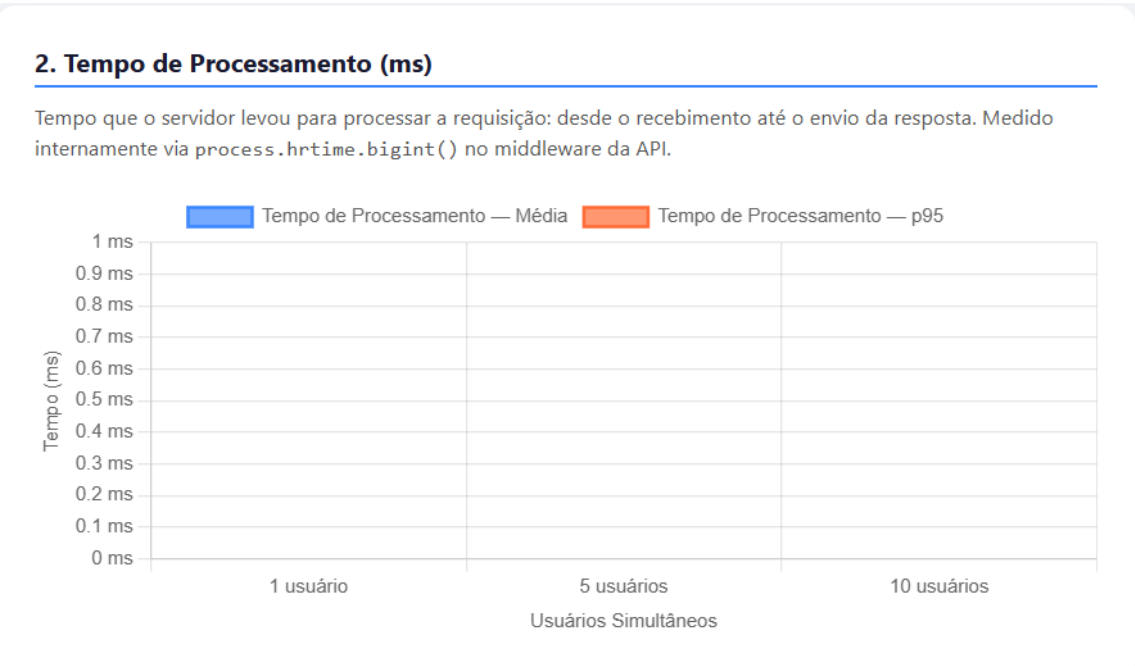
1. Latência (ms)

Tempo de transmissão na rede: intervalo entre o cliente enviar a requisição e o servidor começar a processá-la, mais o retorno. Calculada como: **Tempo de resposta – Tempo de processamento**.



O gráfico de latência demonstra o crescimento do tempo de transmissão de rede conforme o número de usuários simultâneos aumenta. Com 1 usuário, a latência é praticamente nula (0,01 ms), pois há recursos suficientes para atender a requisição imediatamente. Com 5 usuários, sobe para 1,11 ms em média, e com 10 usuários chega a 2,52 ms. Esse comportamento é esperado em ambientes locais, onde a latência de rede é mínima e o principal fator de aumento é a concorrência pelo processamento.

Gráfico 2 — Tempo de Processamento

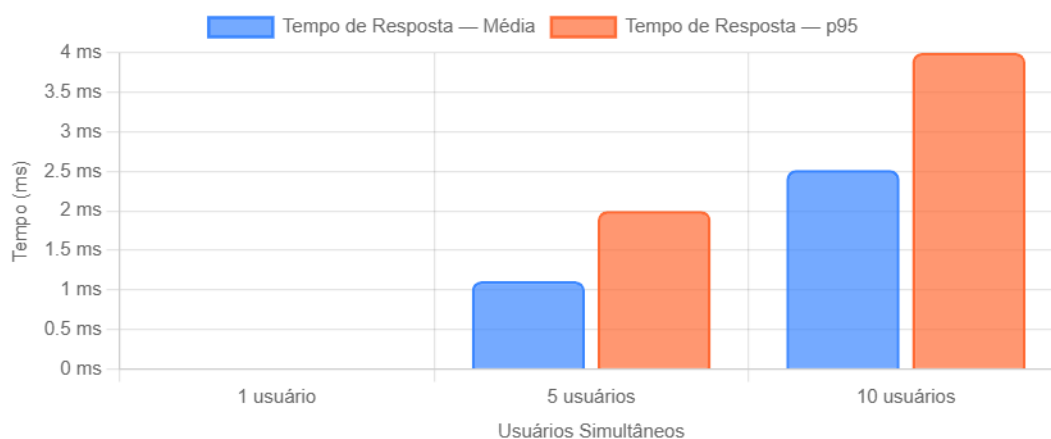


O tempo de processamento manteve-se inferior a 0,01 ms em todos os cenários, demonstrando a eficiência do servidor Node.js com Express no tratamento das requisições. Esse resultado reflete a capacidade do event loop do Node.js de lidar com múltiplas requisições de forma não bloqueante, além da eficiência das consultas realizadas pelo Prisma ORM ao banco de dados MySQL.

Gráfico 3 — Tempo de Resposta

3. Tempo de Resposta (ms)

Tempo total percebido pelo usuário: do envio da requisição até o recebimento completo da resposta. Engloba latência de rede + tempo de processamento. Medido pela ferramenta autocannon no lado cliente.



O tempo de resposta total acompanha o comportamento da latência, uma vez que o tempo de processamento foi negligenciável em todos os cenários. Com 1 usuário, o sistema respondeu em 0,01 ms em média. Com 10 usuários simultâneos, o tempo de resposta médio chegou a 2,52 ms, mantendo-se muito abaixo de qualquer limiar perceptível ao usuário (geralmente considerado acima de 100 ms). O sistema não apresentou nenhum erro em nenhum dos três cenários, totalizando mais de 127.617 requisições processadas com sucesso.

Conclusão

O sistema Aerocode foi desenvolvido com sucesso como uma aplicação web completa, crítica e escalável para o gerenciamento da produção de aeronaves. O back-end foi implementado em Node.js com TypeScript, seguindo princípios de Clean Architecture, com API REST organizada em controllers, services e repositories, integrada ao banco de dados MySQL via Prisma ORM.

Os resultados dos testes de qualidade demonstram que o sistema apresenta excelente desempenho:

- **Zero erros** em todos os cenários de teste
- **Tempo de resposta inferior a 3 ms** mesmo com 10 usuários simultâneos

- **Mais de 5.700 requisições por segundo** com 1 usuário
- **Tempo de processamento sub-milissegundo** em todos os cenários

Esses resultados comprovam que a aplicação atende aos requisitos de qualidade esperados para um sistema crítico, conforme definido pela consultoria descrita no enunciado do trabalho. O sistema foi projetado para rodar nas plataformas Windows 10 ou superior, Linux Ubuntu 24.04.03 ou superior e distribuições derivadas do Ubuntu, garantindo portabilidade e consistência de comportamento em diferentes ambientes.